第21章 错误处理与调试

本章内容

- 理解浏览器错误报告
- 处理错误
- 调试 JavaScript 代码

JavaScript 一直以来被认为是最难调试的编程语言之一，因为它是动态的，且多年来没有适当的开发工具。错误经常会以令人迷惑的浏览器消息形式抛出，比如`"object expected"`。这样的消息没有上下文，因此很难理解。ECMAScript 第 3 版致力于改进这个方面，引入了 `try/catch` 和 `throw` 语句，以及一些错误类型，以帮助开发者在出错时正确地处理它们。几年后，JavaScript 调试器和排错工具开始在浏览器中出现。到了 2008 年，大多数浏览器支持一些 JavaScript 调试能力。

有了适当的语言和开发工具，Web 开发者如今已可以实现适当的错误处理并找到问题的原因。

21.1 浏览器错误报告

所有主流桌面浏览器，包括 IE/Edge、Firefox、Safari、Chrome 和 Opera，都提供了向用户报告错误的机制。默认情况下，所有浏览器都会隐藏错误信息。一个原因是除了开发者之外这些信息对别人没什么用，另一个原因是网页在正常操作中报错的固有特性。

21.1.1 桌面控制台

所有现代桌面浏览器都会通过控制台暴露错误。这些错误可以显示在开发者工具内嵌的控制台中。在前面提到的所有浏览器中，访问开发者工具的路径是相似的。可能最简单的查看错误的方式就是在页面上单击鼠标右键，然后在上下文菜单中选择 Inspect（检查）或 Inspect Element（检查元素），然后再单击 Console（控制台）选项卡。

要直接进入控制台，不同操作系统和浏览器支持不同的快捷键，如下表所示。

浏　览　器	Windows/Linux	Mac
Chrome	Ctrl+Shfit+J	Cmd+Opt+J
Firefox	Ctrl+Shfit+K	Cmd+Opt+K
IE/Edge	F12，然后 Ctrl+2	不适用
Opera	Ctrl+Shift+I	Cmd+Opt+I
Safari	不适用	Cmd+Opt+C

21.1.2　移动控制台

移动浏览器不会直接在设备上提供控制台界面。不过，还是有一些途径可以在移动设备中检查错误。

Chrome 移动版和 Safari 的 iOS 版内置了实用工具，支持将设备连接到宿主操作系统中相同的浏览器。然后，就可以在对应的桌面浏览器中查看错误了。这涉及设备之间的硬件连接，且要遵循不同的操作步骤，比如 Chrome 的操作步骤参见 Google Developers 网站的文章《Android 设备的远程调试入门》，Safari 的操作步骤参见 Apple Developer 网站的文章“Safari Web Inspector Guide”。

此外也可以使用第三方工具直接在移动设备上调试。Firefox 常用的调试工具是 Firebug Lite，这需要通过 JavaScript 的书签小工具向当前页面中加入 Firebug 脚本才可以。脚本运行后，就可以直接在移动浏览器上打开调试界面。Firebug Lite 也有面向其他浏览器（如 Chrome）的版本。

21.2　错误处理

错误处理在编程中的重要性毋庸置疑。所有主流 Web 应用程序都需要定义完善的错误处理协议，大多数优秀的应用程序有自己的错误处理策略，尽管主要逻辑是放在服务器端的。事实上，服务器端团队通常会花很多精力根据错误类型、频率和其他重要指标来定义规范的错误日志机制。最终实现通过简单的数据库查询或报告生成脚本就可以了解应用程序的运行状态。

错误处理在应用程序的浏览器端进展较慢，尽管其重要性一点也不低。这里有一个重要的事实：大多数上网的人没有技术背景，甚至连什么是浏览器都不十分清楚，而且有的人不知道自己使用的是什么浏览器。如前所述，当网页中的 JavaScript 脚本发生错误时，不同浏览器的处理方式不同。不过浏览器处理 JavaScript 报告错误的默认方式对用户并不友好。最好的情况是用户自己不知道发生了什么，然后再重试；最坏的情况是用户感觉特别厌烦，于是永远不回来了。有一个良好的错误处理策略可以让用户知道到底发生了什么。为此，必须理解各种捕获和处理 JavaScript 错误的方式。

21.2.1　`try/catch` 语句

ECMA-262 第 3 版新增了 `try/catch` 语句，作为在 JavaScript 中处理异常的一种方式。基本的语法如下所示，跟 Java 中的 `try/catch` 语句一样：

```
try {
  // 可能出错的代码
} catch (error) {
  // 出错时要做什么
}
```

任何可能出错的代码都应该放到 `try` 块中，而处理错误的代码则放在 `catch` 块中，如下所示：

```
try {
  window.someNonexistentFunction();
} catch (error){
  console.log("An error happened!");
}
```

如果 `try` 块中有代码发生错误，代码会立即退出执行，并跳到 `catch` 块中。`catch` 块此时接收到一个对象，该对象包含发生错误的相关信息。与其他语言不同，即使在 `catch` 块中不使用错误对象，也必须为它定义名称。错误对象中暴露的实际信息因浏览器而异，但至少包含保存错误消息的 `message` 属性。ECMA-262 也指定了定义错误类型的 `name` 属性，目前所有浏览器中都有这个属性。因此，可以

像下面的代码这样在必要时显示错误消息：

```
try {
  window.someNonexistentFunction();
} catch (error){
  console.log(error.message);
}
```

这个例子使用 message 属性向用户显示错误消息。message 属性是唯一一个在 IE、Firefox、Safari、Chrome 和 Opera 中都有的属性，尽管每个浏览器添加了其他属性。IE 添加了 description 属性（其值始终等于 message）和 number 属性（它包含内部错误号）。Firefox 添加了 fileName、lineNumber 和 stack（包含栈跟踪信息）属性。Safari 添加了 line（行号）、sourceId（内部错误号）和 sourceURL 属性。同样，为保证跨浏览器兼容，最好只依赖 message 属性。

1. finally 子句

try/catch 语句中可选的 finally 子句始终运行。如果 try 块中的代码运行完，则接着执行 finally 块中的代码。如果出错并执行 catch 块中的代码，则 finally 块中的代码仍执行。try 或 catch 块无法阻止 finally 块执行，包括 return 语句。比如：

```
function testFinally(){
  try {
    return 2;
  } catch (error){
    return 1;
  } finally {
    return 0;
  }
}
```

这个函数在 try/catch 语句的各个部分都只放了一个 return 语句。看起来该函数应该返回 2，因为它在 try 块中，不会导致错误。但是，finally 块的存在导致 try 块中的 return 语句被忽略。因此，无论什么情况下调用该函数都会返回 0。如果去掉 finally 子句，该函数会返回 2。如果写出 finally 子句，catch 块就成了可选的（它们两者中只有一个是必需的）。

> **注意** 只要代码中包含了 finally 子句，try 块或 catch 块中的 return 语句就会被忽略，理解这一点很重要。在使用 finally 时一定要仔细确认代码的行为。

2. 错误类型

代码执行过程中会发生各种类型的错误。每种类型都会对应一个错误发生时抛出的错误对象。ECMA-262 定义了以下 8 种错误类型：

- Error
- InternalError
- EvalError
- RangeError
- ReferenceError
- SyntaxError
- TypeError
- URIError

Error 是基类型，其他错误类型继承该类型。因此，所有错误类型都共享相同的属性（所有错误对

象上的方法都是这个默认类型定义的方法）。浏览器很少会抛出 Error 类型的错误，该类型主要用于开发者抛出自定义错误。

InternalError 类型的错误会在底层 JavaScript 引擎抛出异常时由浏览器抛出。例如，递归过多导致了栈溢出。这个类型并不是代码中通常要处理的错误，如果真发生了这种错误，很可能代码哪里弄错了或者有危险了。

EvalError 类型的错误会在使用 eval()函数发生异常时抛出。ECMA-262 规定，“如果 eval 属性没有被直接调用（即没有将其名称作为一个 Identifier，也就是 CallExpression 中的 MemberExpression），或者如果 eval 属性被赋值”，就会抛出该错误。基本上，只要不把 eval()当成函数调用就会报告该错误：

```
new eval(); // 抛出 EvalError
eval = foo; // 抛出 EvalError
```

实践中，浏览器不会总抛出 EvalError。Firefox 和 IE 在上面第一种情况下抛出 TypeError，在第二种情况下抛出 EvalError。为此，再加上代码中不大可能这样使用 eval()，因此几乎遇不到这种错误。

RangeError 错误会在数值越界时抛出。例如，定义数组时如果设置了并不支持的长度，如-20 或 Number.MAX_VALUE，就会报告该错误：

```
let items1 = new Array(-20);              // 抛出 RangeError
let items2 = new Array(Number.MAX_VALUE); // 抛出 RangeError
```

RangeError 在 JavaScript 中发生得不多。

ReferenceError 会在找不到对象时发生。（这就是著名的"object expected"浏览器错误的原因。）这种错误经常是由访问不存在的变量而导致的，比如：

```
let obj = x; // 在 x 没有声明时会抛出 ReferenceError
```

SyntaxError 经常在给 eval()传入的字符串包含 JavaScript 语法错误时发生，比如：

```
eval("a ++ b"); // 抛出 SyntaxError
```

在 eval()外部，很少会用到 SyntaxError。这是因为 JavaScript 代码中的语法错误会导致代码无法执行。

TypeError 在 JavaScript 中很常见，主要发生在变量不是预期类型，或者访问不存在的方法时。很多原因可能导致这种错误，尤其是在使用类型特定的操作而变量类型不对时。下面是几个例子：

```
let o = new 10;                           // 抛出 TypeError
console.log("name" in true);              // 抛出 TypeError
Function.prototype.toString.call("name"); // 抛出 TypeError
```

在给函数传参数之前没有验证其类型的情况下，类型错误频繁发生。

最后一种错误类型是 URIError，只会在使用 encodeURI()或 decodeURI()但传入了格式错误的 URI 时发生。这个错误恐怕是 JavaScript 中难得一见的错误了，因为上面这两个函数非常稳健。

不同的错误类型可用于为异常提供更多信息，以便实现适当的错误处理逻辑。在 try/catch 语句的 catch 块中，可以使用 instanceof 操作符确定错误的类型，比如：

```
try {
  someFunction();
} catch (error){
  if (error instanceof TypeError){
```

```
    // 处理类型错误
  } else if (error instanceof ReferenceError){
    // 处理引用错误
  } else {
    // 处理所有其他类型的错误
  }
}
```

检查错误类型是以跨浏览器方式确定适当操作过程的最简单方法，因为 message 属性中包含的错误消息因浏览器而异。

3. try/catch 的用法

当 try/catch 中发生错误时，浏览器会认为错误被处理了，因此就不会再使用本章前面提到的机制报告错误。如果应用程序的用户不懂技术，那么他们即使看到错误也看不懂，这是一个理想的结果。使用 try/catch 可以针对特定错误类型实现自定义的错误处理。

try/catch 语句最好用在自己无法控制的错误上。例如，假设你的代码中使用了一个大型 JavaScript 库的某个函数，而该函数可能会有意或由于出错而抛出错误。因为不能修改这个库的代码，所以为防止这个函数报告错误，就有必要通过 try/catch 语句把该函数调用包装起来，对可能的错误进行处理。

如果你明确知道自己的代码会发生某种错误，那么就不适合使用 try/catch 语句。例如，如果给函数传入字符串而不是数值时就会失败，就应该检查该函数的参数类型并采取相应的操作。这种情况下，没有必要使用 try/catch 语句。

21.2.2 抛出错误

与 try/catch 语句对应的一个机制是 throw 操作符，用于在任何时候抛出自定义错误。throw 操作符必须有一个值，但值的类型不限。下面这些代码都是有效的：

```
throw 12345;
throw "Hello world!";
throw true;
throw { name: "JavaScript" };
```

使用 throw 操作符时，代码立即停止执行，除非 try/catch 语句捕获了抛出的值。

可以通过内置的错误类型来模拟浏览器错误。每种错误类型的构造函数都只接收一个参数，就是错误消息。下面看一个例子：

```
throw new Error("Something bad happened.");
```

以上代码使用一个自定义的错误消息生成了一个通用错误。浏览器会像处理自己生成的错误一样来处理这个自定义错误。换句话说，浏览器会像通常一样报告这个错误，最终显示这个自定义错误。当然，使用特定的错误类型也是一样的，如以下代码所示：

```
throw new SyntaxError("I don't like your syntax.");
throw new InternalError("I can't do that, Dave.");
throw new TypeError("What type of variable do you take me for?");
throw new RangeError("Sorry, you just don't have the range.");
throw new EvalError("That doesn't evaluate.");
throw new URIError("Uri, is that you?");
throw new ReferenceError("You didn't cite your references properly.");
```

自定义错误常用的错误类型是 Error、RangeError、ReferenceError 和 TypeError。

此外，通过继承 Error（第 6 章介绍过继承）也可以创建自定义的错误类型。创建自定义错误类型时，需要提供 name 属性和 message 属性，比如：

```
class CustomError extends Error {
  constructor(message) {
    super(message);
    this.name = "CustomError";
    this.message = message;
  }
}

throw new CustomError("My message");
```

继承 Error 的自定义错误类型会被浏览器当成其他内置错误类型。自定义错误类型有助于在捕获错误时更准确地区分错误。

1. 何时抛出错误

抛出自定义错误是解释函数为什么失败的有效方式。在出现已知函数无法正确执行的情况时就应该抛出错误。换句话说，浏览器会在给定条件下执行该函数时抛出错误。例如，下面的函数会在参数不是数组时抛出错误：

```
function process(values){
  values.sort();

  for (let value of values){
    if (value > 100){
      return value;
    }
  }

  return -1;
}
```

如果给这个函数传入字符串，调用 sort()函数就会失败。每种浏览器对此都会给出一个模棱两可的错误消息，如下所示。

- IE：属性或方法不存在。
- Firefox：values.sort()不是函数。
- Safari：值 undefined（对表达式 values.sort 求值的结果）不是一个对象。
- Chrome：对象名没有方法'sort'。
- Opera：类型不匹配（通常是在需要对象时使用了非对象值）。

虽然 Firefox、Chrome 和 Safari 至少给出了导致错误的相关代码，但并没有哪个错误消息特别明确地指出发生了什么，或者怎么修复。对于上面的一个函数来说，通过这样的错误消息调试还是很容易的。但是，如果是一个复杂的 Web 应用程序，有几千行 JavaScript 代码，想要找到错误的原因就会很难。

这时候，使用适当的信息创建自定义错误可以有效提高代码的可维护性。比如下面的例子：

```
function process(values){
  if (!(values instanceof Array)){
    throw new Error("process(): Argument must be an array.");
  }

  values.sort();
```

图灵社区会员 aSINKz(1561821892@qq.com) 专享 尊重版权

```
  for (let value of values){
    if (value > 100){
      return value;
    }
  }

  return -1;
}
```

在这个重写后的函数中，如果 values 参数不是数组就会抛出错误。错误消息包含函数名以及对错误原因非常清晰的描述。即使在复杂的应用程序中出现这个错误，也可以很容易理解问题所在。

实际编写 JavaScript 代码时，应该仔细评估每个函数，以及可能导致它们失败的情形。良好的错误处理协议可以保证只会发生你自己抛出的错误。

2. 抛出错误与 try/catch

一个常见的问题是何时抛出错误，何时使用 try/catch 捕获错误。一般来说，错误要在应用程序架构的底层抛出，在这个层面上，人们对正在进行的流程知之甚少，因此无法真正地处理错误。如果你在编写一个可能用于很多应用程序的 JavaScript 库，或者一个会在应用程序的很多地方用到的实用函数，那么应该认真考虑抛出带有详细信息的错误。然后捕获和处理错误交给应用程序就行了。

至于抛出错误与捕获错误的区别，可以这样想：应该只在确切知道接下来该做什么的时候捕获错误。捕获错误的目的是阻止浏览器以其默认方式响应；抛出错误的目的是为错误提供有关其发生原因的说明。

21.2.3 error 事件

任何没有被 try/catch 语句处理的错误都会在 window 对象上触发 error 事件。该事件是浏览器早期支持的事件，为保持向后兼容，很多浏览器保持了其格式不变。在 onerror 事件处理程序中，任何浏览器都不会传入 event 对象。相反，会传入 3 个参数：错误消息、发生错误的 URL 和行号。大多数情况下，只有错误消息有用，因为 URL 就是当前文档的地址，而行号可能指嵌入 JavaScript 或外部文件中的代码。另外，onerror 事件处理程序需要使用 DOM Level 0 技术来指定，因为它不遵循 DOM Level 2 Events 标准格式：

```
window.onerror = (message, url, line) => {
  console.log(message);
};
```

在任何错误发生时，无论是否是浏览器生成的，都会触发 error 事件并执行这个事件处理程序。然后，浏览器的默认行为就会生效，像往常一样显示这条错误消息。可以返回 false 来阻止浏览器默认报告错误的行为，如下所示：

```
window.onerror = (message, url, line) => {
  console.log(message);
  return false;
};
```

通过返回 false，这个函数实际上就变成了整个文档的 try/catch 语句，可以捕获所有未处理的运行时错误。这个事件处理程序应该是处理浏览器报告错误的最后一道防线。理想情况下，最好永远不要用到。适当使用 try/catch 语句意味着不会有错误到达浏览器这个层次，因此也就不会触发 error 事件。

> **注意**　浏览器在使用这个事件处理错误时存在明显差异。在 IE 中发生 error 事件时，正常代码会继续执行，所有变量和数据会保持，且可以在 onerror 事件处理程序中访问。然而在 Firefox 中，正常代码会执行会终止，错误发生之前的所有变量和数据会被销毁，导致很难真正分析处理错误。

图片也支持 error 事件。任何时候，如果图片 src 属性中的 URL 没有返回可识别的图片格式，就会触发 error 事件。这个事件遵循 DOM 格式，返回一个以图片为目标的 event 对象。下面是个例子：

```
const image = new Image();

image.addEventListener("load", (event) => {
  console.log("Image loaded!");
});
image.addEventListener("error", (event) => {
  console.log("Image not loaded!");
});

image.src = "doesnotexist.gif"; // 不存在，资源会加载失败
```

在这个例子中，图片加载失败后会显示一个 alert 警告框。这里的关键在于，当 error 事件发生时，图片下载过程已结束，不会再恢复。

21.2.4　错误处理策略

过去，Web 应用程序的错误处理策略基本上是在服务器上落地。错误处理策略涉及很多错误和错误处理考量，包括日志记录和监控系统。这些主要是为了分析模式，以期找到问题的根源并了解有多少用户会受错误影响。

在 Web 应用程序的 JavaScipt 层面落地错误处理策略同样重要。因为任何 JavaScript 错误都可能导致网页无法使用，所以理解这些错误会在什么情况下发生以及为什么会发生非常重要。绝大多数 Web 应用程序的用户不懂技术，在碰到页面出问题时通常会迷惑。为解决问题，他们可能会尝试刷新页面，也可能会直接放弃。作为开发者，应该非常清楚自己的代码在什么情况下会失败，以及失败会导致什么结果。另外，还要有一个系统跟踪这些问题。

21.2.5　识别错误

错误处理非常重要的部分是首先识别错误可能会在代码中的什么地方发生。因为 JavaScript 是松散类型的，不会验证函数参数，所以很多错误只有在代码真正运行起来时才会出现。通常，需要注意 3 类错误：

- 类型转换错误
- 数据类型错误
- 通信错误

上面这几种错误会在特定情况下，在没有对值进行充分检测时发生。

1. 静态代码分析器

不得不说的是，通过在代码构建流程中添加静态代码分析或代码检查器（linter），可以预先发现非常多的错误。这样的代码分析工具有很多，详见 GitHub Gist 网站 All Gists 页面。常用的静态分析工具

是 JSHint、JSLint、Google Closure 和 TypeScript。

静态代码分析器要求使用类型、函数签名及其他指令来注解 JavaScript，以此描述程序如何在基本可执行代码之外运行。分析器会比较注解和 JavaScript 代码的各个部分，对在实际运行时可能出现的潜在不兼容问题给出提醒。

> **注意** 随着代码数量的增长，代码分析器会变得越来越重要，尤其是协作开发者也在增加的情况下。所有主流技术公司都有着庞大的 JavaScript 库，并会在构建流程中使用稳健的静态分析工具。

2. 类型转换错误

类型转换错误的主要原因是使用了会自动改变某个值的数据类型的操作符或语言构造。使用等于（`==`）或不等于（`!=`）操作符，以及在 `if`、`for` 或 `while` 等流控制语句中使用非布尔值，经常会导致类型转换错误。

第 3 章曾讨论过，相等和不相等操作符会自动把执行比较的两个不同类型的值转换为相同类型。在非动态语言中，符号之间是直接比较的，因此很多开发者在 JavaScript 中也会以相同方式来错误地比较值。大多数情况下，最好使用严格相等（`===`）和严格不相等（`!==`）操作符来避免类型转换。来看下面的例子：

```
console.log(5 == "5");   // true
console.log(5 === "5");  // false
console.log(1 == true);  // true
console.log(1 === true); // false
```

这个例子分别使用了相等和严格相等操作符比较了数值 `5` 和字符串 `"5"`。相等操作符会把字符串 `"5"` 转换为数值 `5`，然后再进行比较，结果是 `true`。严格相等操作符发现两个值的数据类型不同，因而直接返回 `false`。同样，对于 `1` 和 `true` 的比较也类似。相等操作符认为它们相等，但严格相等操作符认为它们不相等。使用严格相等和严格不相等操作符可以避免比较过程的类型转换错误，强烈推荐用它们代替相等和不相等操作符。

> **注意** 代码风格指南通常会指出什么时候应使用`===`，什么时候应使用`==`。有些风格指南认同只要始终使用`===`，类型转换就不再是个问题。另一些则认为除了可能发生字符串/布尔值转换的情形，在其他时候使用`===`均是用力过猛的表现。

类型转换错误也会发生在流控制语句中。比如，`if` 语句会自动把条件表达式转换为布尔值，然后再决定下一步的走向。在实践中，`if` 语句是问题比较多的。来看下面的例子：

```
function concat(str1, str2, str3) {
  let result = str1 + str2;
  if (str3) { // 不要!
    result += str3;
  }
  return result;
}
```

这个函数的用意是把两个或三个字符串拼接起来并返回结果。第三个字符串是可选的，因此必须检测它是否存在。如第 3 章所说，命名变量如果没有被赋值就会自动被赋予 `undefined` 值。而在默认转

图灵社区会员 aSINKz(1561821892@qq.com) 专享 尊重版权

换中，undefined 会被转换为布尔值 false。因此这个函数的用意是在提供了第三个参数的情况下，才会在拼接时带上它。问题在于并非只有undefined会转换为false,字符串也不是唯一可转换为true的值。假如第三个参数是数值 0，if 条件判断就会失败，而数值 1 则会导致满足条件。

在流控制语句中使用非布尔值作为条件是很常见的错误来源。为避免这类错误，需要始终坚持使用布尔值作为条件。这通常可以借助某种比较来实现。例如，可以把前面的函数改写为如下形式：

```
function concat(str1, str2, str3){
  let result = str1 + str2;
  if (typeof str3 === "string") { // 恰当的比较
    result += str3;
  }
  return result;
}
```

在这个重写的版本中，if 语句的条件会基于比较操作返回布尔值。这个函数相对更安全，受错误值影响的可能性也更小。

3. 数据类型错误

因为 JavaScript 是松散类型的，所以变量和函数参数都不能保证会使用正确的数据类型。开发者需要自己检查数据类型，确保不会发生错误。数据类型错误常发生在将意外值传给函数的时候。

在前面的例子中，代码检查了第三个参数的数据类型，以确保它是字符串，但根本没有检查另外两个参数。如果函数必须返回一个字符串，那么只传入两个数值，忽略第三个参数就会破坏约定。下面的函数也存在类似问题：

```
// 不安全的函数，任何非字符串值都会导致错误
function getQueryString(url) {
   const pos = url.indexOf("?");
   if (pos > -1){
     return url.substring(pos +1);
   }
   return "";
}
```

这个函数的用途是返回给定 URL 的查询字符串。为此，它先用 indexOf()在字符串中寻找问号，如果找到则使用 substring()方法返回问号后面的所有内容。这两个方法都是只有字符串才有的，因此传入其他类型的值就会导致错误。下面的简单类型检查可以保证函数少出错：

```
function getQueryString(url) {
  if (typeof url === "string") { // 通过类型检查保证安全
    let pos = url.indexOf("?");
    if (pos > -1) {
      return url.substring(pos +1);
    }
  }
  return "";
}
```

在这个重写的版本中，第一步检查了传入的值确实是字符串。这样可以保证函数永远不会因为非字符串值而出错。

如上一节所述，因为存在类型转换，所以应该避免在流控制语句中使用非布尔值作为条件。另外这也是可能导致类型错误的一个做法。来看下面的函数：

```
// 不安全的函数，非数组值可能导致错误
function reverseSort(values) {
  if (values) { // 不要!
    values.sort();
    values.reverse();
  }
}
```

reverseSort()函数可以使用数组的 sort()和 reverse()方法，将数组反向排序。由于 if 语句中的控制条件，任何非数组值都会被转换为 true，从而导致错误。另一个常见的错误是将参数与 null 比较，比如：

```
// 还是不安全的函数，非数组值可能导致错误
function reverseSort(values) {
  if (values != null){ // 不要!
    values.sort();
    values.reverse();
  }
}
```

用参数值与 null 比较只会保证不是两个值：null 和 undefined（对于使用相等和不相等操作符而言是等价的）。与 null 比较不足以保证适当的值，因此不要使用这种方式。出于同样的原因，也不推荐与 undefined 比较。

另一个错误的做法是在检测特性时只检查使用的特性。下面是一个例子：

```
// 仍是不安全的函数，非数组值可能导致错误
function reverseSort(values) {
  if (typeof values.sort === "function") { // 不要!
    values.sort();
    values.reverse();
  }
}
```

在这个例子中，代码检查了参数上是否存在 sort()方法。假如传入的参数确实有一个 sort()方法，但参数本身不是数组，那么在执行 reverse()时也会报告错误。如果知道预期的确切类型，那么最好使用 instanceof 来确定值的正确类型，如下所示：

```
// 安全，非数组值被忽略
function reverseSort(values) {
  if (values instanceof Array) { // 修复
    values.sort();
    values.reverse();
  }
}
```

最后一个 reverseSort()是安全的，它测试了 values 参数是不是 Array 的实例。这样，函数可以保证忽略非数组参数。

一般来说，原始类型的值应该使用 typeof 检测，而对象值应该使用 instanceof 检测。根据函数的用法，不一定要检查每个参数的数据类型，但对外的任何 API 都应该做类型检查以保证正确执行。

4. 通信错误

随着 Ajax 编程的出现，Web 应用程序在运行期间动态加载数据和功能成为常见的情形。JavaScript 和服务器之间的通信也会出现错误。

第一种错误是 URL 格式或发送数据的格式不正确。通常，在把数据发送到服务器之前没有用

encodeURIComponent()编码，会导致这种错误。例如，下面的 URL 格式就不正确：

```
http://www.yourdomain.com/?redir=http://www.someotherdomain.com?a=b&c=d
```

这个 URL 可以通过用 encodeURIComponent()编码"redir="后面的内容来修复，得到的结果如下所示：

```
http://www.example.com/?redir=http%3A%2F%2Fwww.someotherdomain.com%3Fa%3Db%26c%3Dd
```

对于查询字符串，应该都要通过 encodeURIComponent()编码。为此，可以专门定义一个处理查询字符串的函数，比如：

```
function addQueryStringArg(url, name, value) {
  if (url.indexOf("?") == -1){
    url += "?";
  } else {
    url += "&";
  }

  url += '${encodeURIComponent(name)=${encodeURIComponent(value)}';
  return url;
}
```

这个函数接收三个参数：要添加查询字符串的 URL、参数名和参数值。如果 URL 不包含问号，则要给它加上一个；否则就要使用和号（&），以便拼接更多参数和值，因为这意味着前面已有其他查询参数了。查询字符串的名和值在被编码之后会被添加到 URL 中。可以像下面这样使用这个函数：

```
const url = "http://www.somedomain.com";
const newUrl = addQueryStringArg(url, "redir",
                                 "http://www.someotherdomain.com?a=b&c=d");
console.log(newUrl);
```

使用这个函数而不是手动构建 URL 可以保证编码合适，以避免相关错误发生。

在服务器响应非预期值时也会发生通信错误。在动态加载脚本或样式时，请求的资源有可能不可用。有些浏览器在没有返回预期资源时会静默失败，而其他浏览器则会报告错误。不过，在动态加载资源的情况下出错，是不太好做错误处理的。有时候，使用 Ajax 通信可能会提供关于错误条件的更多信息。

21.2.6　区分重大与非重大错误

任何错误处理策略中一个非常重要的方面就是确定某个错误是否为重大错误。具有以下一个或多个特性的错误属于非重大错误：

- 不会影响用户的主要任务；
- 只会影响页面中某个部分；
- 可以恢复；
- 重复操作可能成功。

本质上，不需要担心非重大错误。例如，Gmail 有一个功能，可以让用户在其界面上发送环聊（Hangouts）消息。如果在某个条件下，环聊功能不工作了，就不能算重大错误，因为这不是应用程序的主要功能。Gmail 主要用于阅读和撰写电子邮件，只要用户可以做到这一点，就没有理由中断用户体验。对于非重大错误，无须明确给用户发送消息。可以将受影响的页面区域替换成一条消息，表示该功能暂时不能使用，但不需要中断用户体验。

另一方面，重大错误具备如下特性：

- 应用程序绝对无法继续运行；
- 错误严重影响了用户的主要目标；
- 会导致其他错误发生。

理解 JavaScript 中何时会发生重大错误极其重要，因为这样才能采取应对措施。当重大错误发生时，应该立即发送消息让用户知晓自己不能再继续使用应用程序了。如果必须刷新页面才能恢复应用程序，那就应该明确告知用户，并提供一个自动刷新页面的按钮。

代码中则不要区分什么是或什么不是重大错误。非重大错误和重大错误的区别主要体现在对用户的影响上。好的代码设计意味着应用程序某个部分的错误不会影响其他部分，实际上根本不应该相关。例如，在个性化的主页上，比如 Gmail，可能包含多个相互独立的功能模块。如果每个模块都通过 JavaScript 调用来初始化，那就可能会在代码中看到以下逻辑：

```
for (let mod of mods){
    mod.init(); // 可能的重大错误
}
```

表面上看，这段代码没什么问题，就是依次调用每个模块的 `init()`方法。问题在于，这里只要有一个模块的 `init()`方法出错，数组中其后的所有模块都不会被初始化。如果错误发生在第一个模块上，页面上就没有模块会被初始化了。逻辑上，这样写代码是不合适的，因为每个模块相互独立，各自功能没有相关性。由此可能导致重大错误的原因是代码的结构。好在可以简单地重写以上代码，让每个模块的错误变成非重大错误：

```
for (let mod of mods){
  try {
    mod.init();
  } catch (ex){
    // 在这里处理错误
  }
}
```

通过在 `for` 循环中加入 `try/catch` 语句，模块初始化过程中的任何错误都不会影响其他模块初始化。如果代码中有错误发生，则可以单独处理，并不会影响用户体验。

21.2.7 把错误记录到服务器中

Web 应用程序开发中的一个常见做法是建立中心化的错误日志存储和跟踪系统。数据库和服务器错误正常写到日志中并按照常用 API 加以分类。对复杂的 Web 应用程序而言，最好也把 JavaScript 错误发送回服务器记录下来。这样做可以把错误记录到与服务器相同的系统，只要把它们归类到前端错误即可。使用相同的系统可以进行相同的分析，而不用考虑错误来源。

要建立 JavaScript 错误日志系统，首先需要在服务器上有页面或入口可以处理错误数据。该页面只要从查询字符串中取得错误数据，然后把它们保存到错误日志中即可。比如，该页面可以使用如下代码：

```
function logError(sev, msg) {
  let img = new Image(),
      encodedSev = encodeURIComponent(sev),
      encodedMsg = encodeURIComponent(msg);
  img.src = 'log.php?sev=${encodedSev}&msg=${encodedMsg}';
}
```

logError()函数接收两个参数：严重程度和错误消息。严重程度可以是数值或字符串，具体取决于使用的日志系统。这里使用 Image 对象发送请求主要是从灵活性方面考虑的。

- 所有浏览器都支持 Image 对象，即使不支持 XMLHttpRequest 对象也一样。
- 不受跨域规则限制。通常，接收错误消息的应该是多个服务器中的一个，而 XMLHttpRequest 此时就比较麻烦。
- 记录错误的过程很少出错。大多数 Ajax 通信借助 JavaScript 库的包装来处理。如果这个库本身出错，而你又要利用它记录错误，那么显然错误消息永远不会发给服务器。

只要是使用 try/catch 语句的地方，都可以把相关错误记录下来。下面是一个例子：

```
for (let mod of mods){
  try {
    mod.init();
  } catch (ex){
    logError("nonfatal", 'Module init failed: ${ex.message}');
  }
}
```

在这个例子中，模块初始化失败就会调用 logError()函数。第一个参数是表示错误严重程度的 "nonfatal"，第二个参数在上下文信息后面追加了 JavaScript 错误消息。记录到服务器的错误消息应该包含尽量多的上下文信息，以便找出错误的确切原因。

21.3　调试技术

在 JavaScript 调试器出现以前，开发者必须使用创造性的方法调试代码。结果就出现了各种各样专门为输出调试信息而设计的代码。其中最为常用的调试技术是在相关代码中插入 alert()，这种方式既费事（调试完之后还得清理）又麻烦（如果有漏洞的警告框出现在产品环境中，会给用户造成不便）。已不再推荐将警告框用于调试，因为有其他更好的解决方案。

21.3.1　把消息记录到控制台

所有主流浏览器都有 JavaScript 控制台，该控制台可用于查询 JavaScript 错误。另外，这些浏览器都支持通过 console 对象直接把 JavaScript 消息写入控制台，这个对象包含如下方法。

- error(*message*)：在控制台中记录错误消息。
- info(*message*)：在控制台中记录信息性内容。
- log(*message*)：在控制台记录常规消息。
- warn(*message*)：在控制台中记录警告消息。

记录消息时使用的方法不同，消息显示的样式也不同。错误消息包含一个红叉图标，而警告消息包含一个黄色叹号图标。可以像下面这样使用控制台消息：

```
function sum(num1, num2){
  console.log('Entering sum(), arguments are ${num1},${num2}');
  console.log("Before calculation");
  const result = num1 + num2;
  console.log("After calculation");

  console.log("Exiting sum()");
  return result;
}
```

在调用 sum()函数时，会有一系列消息输出到控制台以辅助调试。

把消息输出到 JavaScript 控制台可以辅助调试代码，但在产品环境下应该删除所有相关代码。这可以在部署时使用代码自动完成清理，也可以手动删除。

> **注意** 相比于使用警告框，打印日志消息是更好的调试方法。这是因为警告框会阻塞代码执行，从而影响对异步操作的计时，进而影响代码的结果。打印日志也可以随意输出任意多个参数并检查对象实例（警告框只能将对象序列化为一个字符串再展示出来，因此经常会看到 Object[Object]。

21.3.2 理解控制台运行时

浏览器控制台是个读取–求值–打印–循环（REPL，read-eval-print-loop），与页面的 JavaScript 运行时并发。这个运行时就像浏览器对新出现在 DOM 中的<script>标签求值一样。在控制台中执行的命令可以像页面级 JavaScript 一样访问全局和各种 API。控制台中可以执行任意数量的代码，与它可能会阻塞的任何页面级代码一样。修改、对象和回调都会保留在 DOM 和运行时中。

JavaScript 运行时会限制不同窗口可以访问哪些内容，因而在所有主流浏览器中都可以选择在哪个窗口中执行 JavaScript 控制台输入。你所执行的代码不会有特权提升，仍会受跨源限制和其他浏览器施加的控制规则约束。

控制台运行时也会集成开发者工具，提供常规 JavaScript 开发中所没有的上下文调试工具。其中一个非常有用的工具是最后点击选择器，所有主流浏览器都会提供。在开发者工具的 Element（元素）标签页内，单击 DOM 树中一个节点，就可以在 Console（控制台）标签页中使用$0 引用该节点的 JavaScript 实例。它就跟普通的 JavaScript 实例一样，因此可以读取属性（如$0.scrollWidth），或者调用成员方法（如$0.remove()）。

21.3.3 使用 JavaScript 调试器

在所有主流浏览器中都可以使用的还有 JavaScript 调试器。ECMAScript 5.1 规范定义了 debugger 关键字，用于调用可能存在的调试功能。如果没有相关的功能，这条语句会被简单地跳过。可以像下面这样使用 debugger 关键字：

```
function pauseExecution(){
  console.log("Will print before breakpoint");
  debugger;
  console.log("Will not print until breakpoint continues");
}
```

在运行时碰到这个关键字时，所有主流浏览器都会打开开发者工具面板，并在指定位置显示断点。然后，可以通过单独的浏览器控制台在断点所在的特定词法作用域中执行代码。此外，还可以执行标准的代码调试器操作（单步进入、单步跳过、继续，等等）。

浏览器也支持在开发者工具的源代码标签页中选择希望设置断点的代码行来手动设置断点（不使用 debugger 关键字）。这样设置的断点与使用 debugger 关键字设置的一样，只是不会在不同浏览器会话之间保持。

图灵社区会员 aSINKz(1561821892@qq.com) 专享 尊重版权

21.3.4　在页面中打印消息

另一种常见的打印调试消息的方式是把消息写到页面中指定的区域。这个区域可以是所有页面中都包含的元素，但仅用于调试目的；也可以是在需要时临时创建的元素。例如，可以定义这样 log()函数：

```
function log(message) {
  // 这个函数的词法作用域会使用这个实例
  // 而不是 window.console
  const console = document.getElementById("debuginfo");
  if (console === null){
    console = document.createElement("div");
    console.id = "debuginfo";
    console.style.background = "#dedede";
    console.style.border = "1px solid silver";
    console.style.padding = "5px";
    console.style.width = "400px";
    console.style.position = "absolute";
    console.style.right = "0px";
    console.style.top = "0px";
    document.body.appendChild(console);
  }
  console.innerHTML += '<p> ${message}</p>';
}
```

在这个 log()函数中，代码先检测是否已创建了调试用的元素。如果没有，就创建一个新 元素并给它添加一些样式，以便与页面其他部分区分出来。此后，再使用 innerHTML 属性把消息写到这个 中。结果就是在页面的一个小区域内显示日志信息。

> **注意**　与在控制台输出消息一样，在页面中输入消息的代码也需要从生产环境中删除。

21.3.5　补充控制台方法

记住使用哪个日志方法（原生的 console.log()和自定义的 log()方法），对开发者来说是一种负担。因为 console 是一个全局对象，所以可以为这个对象添加方法，也可以用自定义的函数重写已有的方法，这样无论在哪里用到的日志打印方法，都会按照自定义的方式行事。

比如，可以这样重新定义 console.log 函数：

```
// 把所有参数拼接为一个字符串，然后打印出结果
console.log = function() {
  // 'arguments'并没有 join 方法，这里先把它转换为数组
  const args = Array.prototype.slice.call(arguments);
  console.log(args.join(', '));
}
```

这样，其他代码调用的将是这个函数，而不是通用的日志方法。这样的修改在页面刷新后会失效，因此只是调试或拦截日志的一个有用而轻量的策略。

21.3.6　抛出错误

如前所述，抛出错误是调试代码的很好方式。如果错误消息足够具体，只要看一眼错误就可以确定原因。好的错误消息包含关于错误原因的确切信息，因此可以减少额外调试的工作量。比如下面的函数：

```
function divide(num1, num2) {
  return num1 / num2;
}
```

这个简单的函数执行两个数的除法，但如果任何一个参数不是数值，则返回 NaN。当 Web 应用程序意外返回 NaN 时，简单的计算可能就会出问题。此时，可以检查每个参数的类型是不是数值，然后再进行计算。来看下面的例子：

```
function divide(num1, num2) {
  if (typeof num1 != "number" || typeof num2 != "number"){
    throw new Error("divide(): Both arguments must be numbers.");
  }
  return num1 / num2;
}
```

这里，任何一个参数不是数值都会抛出错误。错误消息中包含函数名和错误的具体原因。当浏览器报告这个错误消息时，你立即就能根据它包含的信息定位到问题，包括问题的解决方案。相对于没那么具体的浏览器错误消息，这个错误消息显示更有价值。

在大型应用程序中，自定义错误通常使用 assert() 函数抛出错误。这个函数接收一个应该为 true 的条件，并在条件为 false 时抛出错误。下面是一个基本的 assert() 函数：

```
function assert(condition, message) {
  if (!condition) {
    throw new Error(message);
  }
}
```

这个 assert() 函数可用于代替多个 if 语句，同时也是记录错误的好地方。下面的代码演示了如何使用它：

```
function divide(num1, num2) {
  assert(typeof num1 == "number" && typeof num2 == "number",
         "divide(): Both arguments must be numbers.");
  return num1 / num2;
}
```

相比于之前的例子，使用 assert() 函数可以减少抛出自定义错误所需的代码量，并且让代码更好理解。

21.4 旧版 IE 的常见错误

IE 曾是最难调试 JavaScript 错误的浏览器之一。该浏览器的旧版本抛出的错误通常比较短，比较含糊，缺少上下文。接下来几节分别讨论旧版 IE 中可能会出现的常见且难于调试的 JavaScript 错误。因为这些浏览器不支持 ES6，所以代码会考虑向后兼容。

21.4.1 无效字符

JavaScript 文件中的代码必须由特定字符构成。在检测到 JavaScript 文件中存在无效字符时，IE 会抛出"invalid character"错误。所谓无效字符，指的是 JavaScript 语法中没有定义过的字符。例如，一个看起来像减号而实际上并不是减号的字符（Unicode 值为\u2013）。这个字符不能用于代替减号（ASCII 码为 45），因为它不是 JavaScript 语法定义的减号。这个特殊字符经常会被自动插入 Word 文档，因此如果把它从 Word 文档复制到文本编辑器然后在 IE 中运行，IE 就会报告文件中包含非法字符。其他

图灵社区会员 aSINKz(1561821892@qq.com) 专享 尊重版权

浏览器也类似，Firefox 抛出"illegal character"错误，Safari 报告语法错误，而 Opera 则报告 ReferenceError（因为把这个字符当成了未定义标识符来解释）。

21.4.2　未找到成员

如前所述，旧版 IE 中所有 DOM 对象都是用 COM 对象实现的，并非原生 JavaScript 对象。在涉及垃圾回收时，这可能会导致很多奇怪的行为。其中，"member not found"错误是 IE 中垃圾回收程序常报告的错误。

这个错误通常会在给一个已被销毁的对象赋值时发生。这个对象必须是 COM 对象才会出现这个消息。最好的一个例子就是 event 对象。IE 的 event 对象是作为 window 的一个属性存在的，会在事件发生时创建，在事件处理程序执行完毕后销毁。因此，如果你想在稍后会执行的闭包中使用 event 对象，尝试给 event 对象赋值就会导致这个错误，如下面的例子所示：

```
document.onclick = function() {
  var event = window.event;
  setTimeout(function(){
    event.returnValue = false; // 未找到成员
  }, 1000);
};
```

在这个例子中，文档被添加了单击事件处理程序。事件处理程序把 window.event 对象保存在一个名为 event 的本地变量中。然后，在传递给 setTimeout()的闭包中引用这个事件变量。当 onclick 事件处理程序退出后，event 对象会被销毁，因此闭包中对它的引用也就不存在了，于是就会报告未找到成员错误。之所以给 event.returnValue 赋值会导致"member not found"错误，是因为不能给已将其成员销毁的 COM 对象赋值。

21.4.3　未知运行时错误

使用 innerHTML 或 outerHTML 属性以下面一种方式添加 HTML 时会发生未知运行时错误：比如将块级元素插入行内元素，或者在表格的任何部分（<table>、<tbody>等）访问了其中一个属性。例如，从技术角度来说，<p>标签不能包含另一个块级元素，如 ，因此以下代码会导致未知运行时错误：

```
p.innerHTML = "<div>Hi</div>"; // where p contains a <p> element
```

在将块级元素插入不恰当的位置时，其他浏览器会尝试纠正，这样就不会发生错误，但 IE 在这种情况下要严格得多。

21.4.4　语法错误

通常，当 IE 报告语法错误时，原因是很清楚的。一般来说，可以通过错误消息追踪到少了一个分号或括号错配。不过，有一种情况下报告的语法错误并不清楚。

如果网页中引用的一个外部 JavaScript 文件由于某种原因返回了非 JavaScript 代码，则 IE 会抛出语法错误。例如，错误地把<script>标签的 src 属性设置为指向一个 HTML 文件，就会导致语法错误。通常会报告该语法错误发生在脚本第一行的第一个字符。Opera 和 Safari 此时也会报告语法错误，但它们也会报告是引用文件不当导致的问题。IE 没有这些信息，因此需要仔细检查引用的每个外部 JavaScript 文件。Firefox 会忽略作为 JavaScript 引用的非 JavaScript 文件导致的解析错误。

这种错误通常发生在服务器端动态生成 JavaScript 的情况下。很多服务器端语言会在发生运行时错误时，自动向输出中插入 HTML。这种输出显然会导致 JavaScript 语法错误。如果你碰到了难以排除的语法错误，可以仔细检查所有外部文件，确保没有文件包含服务器由于错误而插入的 HTML。

21.4.5 系统找不到指定资源

还有一个可能最没用的消息："The system cannot locate the resource specified"（系统找不到指定资源）。这个错误会在 JavaScript 向某个 URL 发送请求，而该 URL 长度超过了 IE 允许的最大 URL 长度（2083 个字符）时发生。这个长度限制不仅针对 JavaScript，而且针对 IE 本身。（其他浏览器没有这么严格地限制 URL 长度。）另外，IE 对 URL 路径还有 2048 个字符的限制。下面的代码会导致这个错误：

```
function createLongUrl(url) {
  var s = "?";
  for (var i = 0, len = 2500; i < len; i++){
    s += "a";
  }

  return url + s;
}

var x = new XMLHttpRequest();
x.open("get", createLongUrl("http://www.somedomain.com/"), true);
x.send(null);
```

在这个例子中，`XMLHttpRequest` 对象尝试向超过 URL 长度限制的地址发送请求。在调用 `open()` 方法时，错误会发生。为避免这种错误，一个办法是缩短请求成功所需的查询字符串，比如缩短参数名或去掉不必要的数据。另一个办法是改为使用 POST 请求，不用查询字符串而通过请求体发送数据。

21.5 小结

对于今天复杂的 Web 应用程序而言，JavaScript 中的错误处理十分重要。未能预测什么时候会发生错误以及如何从错误中恢复，会导致糟糕的用户体验，甚至造成用户流失。大多数浏览器默认不向用户报告 JavaScript 错误，因此在开发和调试时需要自己实现错误报告。不过在生产环境中，不应该以这种方式报告错误。

下列方法可用于阻止浏览器对 JavaScript 错误作出反应。

- 使用 `try/catch` 语句，可以通过更合适的方式对错误做出处理，避免浏览器处理。
- 定义 `window.onerror` 事件处理程序，所有没有通过 `try/catch` 处理的错误都会被该事件处理程序接收到（仅限 IE、Firefox 和 Chrome）。

开发 Web 应用程序时，应该认真考虑可能发生的错误，以及如何处理这些错误。

- 首先，应该分清哪些算重大错误，哪些不算重大错误。
- 然后，要通过分析代码预测很可能发生哪些错误。由于以下因素，JavaScript 中经常出现错误：
  - 类型转换；
  - 数据类型检测不足；
  - 向服务器发送错误数据或从服务器接收到错误数据。

IE、Firefox、Chrome、Opera 和 Safari 都有 JavaScript 调试器，有的内置在浏览器中，有的是作为扩展，需另行下载。所有调试器都能够设置断点、控制代码执行和在运行时检查变量值。